

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 20%

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I **JAIVANT.S (192311326)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **JAIVANT.S**

Assessment Tasks

Case Study

1. Case 1: A food delivery company wants to deploy a cloud-native backend with web APIs, browser clients, and shared cloud storage for receipts and delivery images. Propose the architecture and explain the role of each component.
2. Case 2: A media startup must decide between SAN-backed storage, object storage, and serverless processing for video upload workflows. Analyze the case and recommend a deployment model.
3. Case 3: A smart city platform needs low-latency decision making near sensors while still using the cloud for analytics. Analyze the use of edge computing and fog computing in this environment.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis

Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

ANSWERS

Case 1: Cloud-Native Backend for a Food Delivery Company

Introduction

A food delivery company needs a system that can handle customers using web browsers, mobile applications, restaurants receiving orders, and delivery partners updating order status. Since the number of users can change significantly, a cloud-native architecture would be suitable because it can scale according to demand.

Proposed Architecture

The proposed architecture can contain the following components:

1. Browser and Mobile Clients

Customers can use web browsers or mobile applications to search restaurants, place orders, make payments, and track deliveries. These clients communicate with the backend through APIs.

2. API Gateway

The API Gateway acts as the main entry point between clients and backend services. It can handle authentication, request routing, rate limiting, and security.

3. Web/API Servers

Backend servers provide APIs for restaurant search, user accounts, orders, payments, and delivery tracking. They process requests received from the clients.

4. Microservices

The application can be divided into services such as:

- User service
- Restaurant service
- Order service
- Payment service
- Delivery service
- Notification service

This makes the application easier to develop, maintain, and scale.

5. Database

A cloud database can store structured information such as customer details, restaurant information, orders, and payment records.

6. Cloud Object Storage

Receipts, restaurant images, delivery photographs, and other files can be stored in cloud object storage. This is more suitable for large files than storing them directly in the application database.

7. Message Queue

A message queue can handle background operations such as sending notifications, updating delivery status, and processing receipts without slowing down the main application.

8. Load Balancer

A load balancer distributes incoming requests among multiple backend servers. This prevents a single server from becoming overloaded.

Working of the Architecture

When a customer places an order, the request first reaches the API Gateway. The gateway forwards it to the appropriate service, such as the Order Service. The order information is stored in the database, while the restaurant and delivery services are notified through a messaging system.

If a receipt or delivery image is generated, it is stored in cloud object storage. The customer can later access the file through a secure URL.

Benefits

- Easy scaling during peak meal hours.
- Better reliability because services can operate independently.
- Secure storage for images and receipts.
- Faster communication through APIs.
- Easier maintenance using separate services.

Conclusion

A cloud-native architecture using **API Gateway, microservices, cloud databases, object storage, load balancing, and messaging services** would provide the food delivery company with a scalable, reliable, and flexible backend.

Case 2: Media Startup – Video Upload Workflow

Introduction

A media startup needs to store and process large video files uploaded by users. Videos require significant storage capacity, and the company may experience unpredictable upload traffic. Therefore, choosing the right storage and processing model is important.

SAN-Backed Storage

A **Storage Area Network (SAN)** provides high-speed block-level storage that can be accessed by servers.

Advantages:

- High performance.
- Suitable for applications requiring fast block-level access.
- Centralized storage management.

Limitations:

- Can be expensive for a startup.
- Scaling storage may require additional infrastructure.
- Not ideal for massive amounts of unstructured video data.

Object Storage

Object storage stores files as objects along with their metadata. Services such as cloud object storage are well suited for videos.

Advantages:

- Highly scalable.
- Suitable for large video files.
- Lower storage and infrastructure management costs.
- Easy access through APIs.
- Supports large amounts of unstructured data.

For a media startup, object storage would be a better choice for storing original videos, thumbnails, and processed video files.

Serverless Processing

Serverless computing allows video-processing functions to run only when required. For example, when a user uploads a video, an event can automatically trigger a serverless function.

The function can initiate:

- Video format conversion.
- Thumbnail generation.
- Metadata extraction.
- Transcoding workflows.
- Content validation.

The startup does not need to maintain dedicated servers for every processing task.

Recommended Deployment Model

The best approach would be a combination of **object storage and serverless processing**.

The workflow can be:

User → Upload Service → Object Storage → Event Trigger → Serverless Processing → Processed Video Storage → Content Delivery

When the user uploads a video, it is stored directly in object storage. The upload event triggers a serverless workflow, which processes the video and stores the final versions back in object storage.

Why Not SAN?

Although SAN provides high performance, it introduces additional cost and infrastructure management. A startup generally benefits more from the scalability and pay-as-you-use nature of cloud object storage.

Benefits of the Recommended Model

- Handles large video files efficiently.
- Automatically scales during heavy uploads.
- Reduces infrastructure management.
- Pay only for the resources used.
- Easy to expand as the startup grows.

Conclusion

For a media startup, **cloud object storage combined with serverless processing** is the most suitable deployment model. SAN can be considered for specialized high-performance workloads, but object storage and serverless services provide better scalability, flexibility, and cost efficiency for typical video upload workflows.

Case 3: Smart City – Edge and Fog Computing

Introduction

A smart city can generate huge amounts of data from traffic cameras, pollution sensors, smart vehicles, surveillance systems, and other IoT devices. Sending all this data directly to the cloud may cause delays and consume significant network bandwidth. Edge and fog computing can solve this problem.

Edge Computing

Edge computing processes data close to where it is generated, such as directly on sensors, cameras, traffic controllers, or nearby gateways.

For example, a traffic camera can identify an accident locally instead of sending the entire video stream to the cloud.

Benefits of Edge Computing

- Very low response time.
- Reduces network bandwidth usage.
- Can operate even with limited connectivity.
- Useful for time-sensitive decisions.
- Improves privacy by processing sensitive information locally.

Fog Computing

Fog computing provides an intermediate processing layer between edge devices and the cloud.

For example, several traffic sensors in a particular area can send their data to a nearby fog node. The fog node processes and combines the information before sending important results to the cloud.

Benefits of Fog Computing

- Provides processing closer to users and sensors.
- Reduces the amount of data sent to the cloud.
- Supports coordination between multiple edge devices.
- Provides faster response than cloud-only processing.
- Can aggregate data from an entire local area.

Role of Cloud Computing

The cloud can be used for tasks that do not require immediate responses, such as:

- Long-term data storage.
- City-wide analytics.
- Historical trend analysis.
- Machine learning model training.
- Generating management reports.

Example of the Combined Architecture

A smart traffic system could work as follows:

Sensors/Cameras → Edge Processing → Fog Node → Cloud Platform

The edge device can immediately detect an accident. The fog node can analyze traffic conditions across several nearby roads. The cloud can store the historical information and use it to predict future traffic patterns.

Edge vs Fog vs Cloud

Feature	Edge	Fog	Cloud
Processing location	Near sensor	Local network/intermediate layer	Central data center
Latency	Very low	Low	Higher
Main purpose	Immediate decisions	Local coordination	Large-scale analytics
Storage	Limited	Moderate	Very high
Example	Accident detection	Area traffic analysis	City-wide traffic prediction

Recommended Approach

The smart city should use a **hybrid Edge–Fog–Cloud architecture** rather than depending completely on one technology. Edge computing should handle immediate decisions, fog computing should coordinate local data, and cloud computing should perform large-scale analytics and long-term storage.

Conclusion

Edge and fog computing are highly suitable for smart cities because they reduce latency and network traffic while still allowing the cloud to perform large-scale analytics. A combination of **Edge + Fog + Cloud** provides a balanced architecture for real-time and long-term smart city applications.
